

Conversation Transcript — SCD Type 2 Implementation Using MERGE

Exported on Friday, September 18, 2026

USER

So in one the interview interviewer asked asked ur implementing SCD TYPE 2 Using merge but how many merge statements are required to do ??

ASSISTANT

Two MERGE statements are typically required — this is the standard pattern for SCD Type 2.

Why one isn't enough: In a single MERGE, when a matching key is found and the data has changed, you need to (a) expire/close the old record (update its `end_date` / `is_current` flag) AND (b) insert a brand-new row with the new values and current flag. A MERGE statement's WHEN MATCHED / WHEN NOT MATCHED logic can only take one action per matched row — it can't update the existing row and insert a new row for that same match in one pass.

The typical 2-MERGE pattern:

1. **First MERGE** — expire changed records
 - WHEN MATCHED AND (data changed) → UPDATE set `end_date = current_date` , `is_current = false`
 - WHEN NOT MATCHED → do nothing (or handle separately)
2. **Second MERGE** (or INSERT) — bring in new versions
 - WHEN NOT MATCHED (by target) → INSERT new row for genuinely new records
 - WHEN NOT MATCHED but exists in a "changed" staging set → INSERT new current row with old key + new attribute values + `start_date = today` , `is_current = true`

Some people implement step 2 as a plain INSERT INTO ... SELECT rather than a MERGE, so you'll sometimes hear it described as "1 MERGE + 1 INSERT" — functionally it's the same idea.

Interviewer follow-up to expect: "Can you do it in one?" — answer: only with a UPSERT + trigger, a staging table plus window functions, or using platform-specific extensions (e.g., Databricks' MERGE combined with a CDC/ foreachBatch pattern) that decouple the expire/insert logic outside pure MERGE syntax. But with vanilla ANSI MERGE, 2 is the accepted minimum.

USER

Then genrate me a detailed SCD 2 implementation with example

ASSISTANT

Here's a full SCD Type 2 implementation using the 2-MERGE pattern, with sample data.

Setup

Target (dimension) table:

```
CREATE TABLE dim_customer (  
  customer_sk      INT IDENTITY(1,1) PRIMARY KEY,  -- surrogate key  
  customer_id      INT,                            -- natural/business key  
  customer_name    VARCHAR(100),  
  address          VARCHAR(200),  
  segment          VARCHAR(50),  
  start_date       DATE,  
  end_date         DATE,  
  is_current       BOOLEAN  
);
```

Source (staging) table: fresh data from upstream system

```
CREATE TABLE stg_customer (  
  customer_id      INT,  
  customer_name    VARCHAR(100),  
  address          VARCHAR(200),  
  segment          VARCHAR(50)  
);
```

Sample initial load:

--	--	--	--	--	--	--	--

customer_sk	customer_id	name	address	segment	start_date	end_date	is_current
1	101	Alice	Delhi	Retail	2024-01-01	9999-12-31	true
2	102	Bob	Mumbai	Retail	2024-01-01	9999-12-31	true

New incoming data (Alice moved to Pune, Bob unchanged, new customer 103):

customer_id	name	address	segment
101	Alice	Pune	Retail
102	Bob	Mumbai	Retail
103	Carol	Chennai	Wholesale

Step 1: MERGE #1 — Expire changed records

```
MERGE INTO dim_customer AS tgt
USING stg_customer AS src
ON tgt.customer_id = src.customer_id
  AND tgt.is_current = true
WHEN MATCHED AND (
    tgt.customer_name <> src.customer_name OR
    tgt.address <> src.address OR
    tgt.segment <> src.segment
)
THEN UPDATE SET
  tgt.end_date = CURRENT_DATE - 1,
  tgt.is_current = false;
```

This only touches Alice's row (address changed) — sets her end_date and flips is_current to false. Bob's row isn't touched since nothing changed.

Step 2: MERGE #2 — Insert new versions + brand-new customers

```
MERGE INTO dim_customer AS tgt
USING stg_customer AS src
ON tgt.customer_id = src.customer_id
  AND tgt.is_current = true
WHEN NOT MATCHED THEN
INSERT (customer_id, customer_name, address, segment, start_date, end_date, is_current)
VALUES (src.customer_id, src.customer_name, src.address, src.segment,
        CURRENT_DATE, '9999-12-31', true);
```

Here's the key trick: after Step 1 ran, Alice's row is no longer is_current = true, so the ON clause fails to match her against any current row — she falls into WHEN NOT MATCHED and gets a fresh row inserted. Carol (new) also falls into WHEN NOT MATCHED for the same reason (no row at all). Bob matches an existing current row with no attribute mismatch, so nothing happens to him.

Result after both MERGEs

customer_sk	customer_id	name	address	segment	start_date	end_date	is_current
1	101	Alice	Delhi	Retail	2024-01-01	2025-XX-XX	false
2	102	Bob	Mumbai	Retail	2024-01-01	9999-12-31	true
3	101	Alice	Pune	Retail	today	9999-12-31	true
4	103	Carol	Chennai	Wholesale	today	9999-12-31	true

Alice now has full history (old Delhi row closed, new Pune row current), Bob is untouched, Carol is added as new.

Key design notes an interviewer may probe

- **Why check is_current = true in the join condition?** Without it, MERGE could match against an already-expired historical row and corrupt history.
- **Hash comparison instead of column-by-column:** In production, people often add a row_hash column (MD5/SHA of tracked attributes) and compare hashes instead of listing every column — cleaner when there are many attributes.
- **Handling deletes (optional SCD2 extension):** A third step marks records missing from source as expired (is_current = false) if you need to track deletions.
- **Order matters:** Step 1 must run before Step 2, since Step 2's "not matched" logic depends on Step 1 having already flipped the changed row's is_current flag.

USER

Then explain in hash way also

ASSISTANT

Here's the same implementation, but using a hash column instead of comparing every attribute individually — this is the pattern most production pipelines actually use.

Why hash instead of column-by-column

With 3 columns, listing them out is fine. With 30 columns, `tgt.col1 <> src.col1 OR tgt.col2 <> src.col2 OR ...` becomes unreadable and error-prone (miss one column, silently break change detection). A single hash of all tracked attributes collapses that into one comparison.

Step 0: Add a hash column

```
ALTER TABLE dim_customer ADD row_hash VARCHAR(64);
```

Compute the hash from all attributes you want to track for changes (never include the surrogate key, dates, or `is_current` in the hash — only business attributes):

```
-- Example hash expression (works in most warehouses, syntax varies)
SHA2(CONCAT_WS('|', customer_name, address, segment), 256)
```

`CONCAT_WS('|', ...)` joins columns with a delimiter before hashing — the delimiter matters, otherwise `('AB','C')` and `('A','BC')` could hash the same if you just concatenated blindly.

Updated staging table (compute hash on load)

```
CREATE TABLE stg_customer AS
SELECT
  customer_id,
  customer_name,
  address,
  segment,
  SHA2(CONCAT_WS('|', customer_name, address, segment), 256) AS row_hash
FROM raw_source_customer;
```

Sample data with hashes (illustrative — real SHA2 is a 64-char hex string, shortened here for readability):

customer_id	name	address	segment	row_hash
101	Alice	Delhi	Retail	a1f9c3... (old)
101	Alice	Pune	Retail	7b2e88... (new — changed)
102	Bob	Mumbai	Retail	d44a01... (unchanged)
103	Carol	Chennai	Wholesale	9e0f76... (new row)

Step 1: MERGE #1 — Expire changed records (hash comparison)

```
MERGE INTO dim_customer AS tgt
USING stg_customer AS src
ON tgt.customer_id = src.customer_id
AND tgt.is_current = true
WHEN MATCHED AND tgt.row_hash <> src.row_hash
THEN UPDATE SET
  tgt.end_date = CURRENT_DATE - 1,
  tgt.is_current = false;
```

Alice's target hash `a1f9c3...` $\neq$ source hash `7b2e88...` → matched, expired.

Bob's target hash `d44a01...` = source hash `d44a01...` → not matched, untouched.

Step 2: MERGE #2 — Insert new/changed versions

```
MERGE INTO dim_customer AS tgt
USING stg_customer AS src
ON tgt.customer_id = src.customer_id
AND tgt.is_current = true
WHEN NOT MATCHED THEN
INSERT (customer_id, customer_name, address, segment, row_hash,
  start_date, end_date, is_current)
VALUES (src.customer_id, src.customer_name, src.address, src.segment, src.row_hash,
  CURRENT_DATE, '9999-12-31', true);
```

Same logic as before — after Step 1, Alice no longer has a current row to match against, so she's treated as "not matched" and gets a new row with her new hash `7b2e88...`. Carol has no existing row at all, same outcome. Bob still matches his current row, so nothing

happens.

Result

customer_sk	customer_id	name	address	row_hash	is_current
1	101	Alice	Delhi	a1f9c3...	false
2	102	Bob	Mumbai	d44a01...	true
3	101	Alice	Pune	7b2e88...	true
4	103	Carol	Chennai	9e0f76...	true

Interview-worthy details

- **Collision risk:** SHA-256 collisions are practically impossible at real-world dimension sizes — this is a non-issue in practice, but interviewers sometimes ask "what if two different rows hash the same?" to see if you understand it's a theoretical, not practical, concern.
- **Deterministic column order:** the hash must always concatenate columns in the same order on both source and target — if the source pipeline changes column order in the `CONCAT_WS`, all rows will appear "changed" even if nothing did.
- **NULL handling:** `CONCAT_WS` in most engines silently skips NULLs rather than erroring, which can cause two genuinely different rows (one with a NULL address, one with empty string) to hash the same — worth explicitly handling with `COALESCE(address, '')` if that distinction matters.
- **Storing the hash lets you skip re-hashing on every run** — you compute it once at ingestion and just compare stored values, which is cheaper than recomputing full column comparisons every MERGE.

USER

tgt.end_date = CURRENT_DATE - 1, why -1?

ASSISTANT

`CURRENT_DATE - 1` keeps the date ranges non-overlapping and contiguous.

The problem it avoids: the new row's `start_date` is `CURRENT_DATE`. If the old row's `end_date` was also set to `CURRENT_DATE`, both rows would technically include `CURRENT_DATE` in their validity range at the same time — a one-day overlap where two versions of the same customer are both "valid" on that date. That breaks point-in-time queries like:

```
SELECT * FROM dim_customer
WHERE customer_id = 101
AND '2025-06-15' BETWEEN start_date AND end_date;
```

If `2025-06-15` is the change date and both rows cover it, you'd get **two rows back instead of one** — ambiguous history.

With the `-1`:

- Old row: `start_date = 2024-01-01`, `end_date = 2025-06-14` (yesterday)
- New row: `start_date = 2025-06-15` (today), `end_date = 9999-12-31`

Every date maps to exactly one row. Clean, no overlap, no gap.

Alternative convention (equally valid, just a different boundary style): some teams instead do half-open intervals — `end_date = CURRENT_DATE` on the old row (inclusive) but query with `start_date <= X AND (end_date > X OR end_date IS NULL)`, i.e., `end_date` is *exclusive*. Either convention works as long as it's applied **consistently** across the whole pipeline and all downstream queries. What you must avoid is inclusive-inclusive on both ends with the same date — that's the actual bug.

Interview tip: if asked "why not just use `end_date = CURRENT_DATE` on old and `start_date = CURRENT_DATE` on new?" — point out that's the half-open version, and it's fine too, but you have to be explicit about which boundary is inclusive/exclusive and make sure `BETWEEN`-style queries elsewhere in the codebase agree with that choice. The `-1` approach is popular precisely because it lets you keep simple inclusive `BETWEEN start_date AND end_date` queries everywhere without ambiguity.